

- Description

This module manages the book borrowing process in a library system. It ensures that:

1. A user **cannot borrow more books than the maximum allowed limit**.
2. The requested number of books **must be available in stock**.
3. If borrowing is successful, the **user's borrowed book count increases**, and the **library's stock decreases**.
4. If borrowing fails, **the book count remains unchanged**.

This module provides a structured approach to managing book borrowing while maintaining numerical constraints.

- Specification

module LibrarySystem

var maxBorrowLimit, userBorrowedBooks, bookStock, numBooks: integer

process borrowBook(numBooks: integer) flag: boolean

functional scenario form:

numBooks <= bookStock and (userBorrowedBooks + numBooks) <= maxBorrowLimit
&& flag = true and userBorrowedBooks = userBorrowedBooks + numBooks and
bookStock = bookStock – numBooks, Valid borrowing, book count updated

||

numBooks > bookStock && flag = false, Invalid borrowing, insufficient stock, values
unchanged

||

userBorrowedBooks + numBooks > maxBorrowLimit && flag = false, Invalid borrowing,
user has reached borrowing limit, values unchanged

||

numBooks <= 0 && flag = false, Invalid borrowing, book count must be greater than
zero, values unchanged

end process

end module

- Java Code

```
package code;
public class c03_LibrarySystem {
    private int maxBorrowLimit;
    private int userBorrowedBooks;
    private int bookStock;
    public c03_LibrarySystem(int maxBorrowLimit, int initialStock) {
        this.maxBorrowLimit = maxBorrowLimit;
        this.userBorrowedBooks = 0;
        this.bookStock = initialStock;
    }
    public boolean borrowBook(int numBooks) {
```

```

        if (numBooks > 0 && numBooks <= bookStock && (userBorrowedBooks
+ numBooks) <= maxBorrowLimit) {
            userBorrowedBooks += numBooks;
            bookStock -= numBooks;
            System.out.printf("Borrow successful! You now have %d books
. Remaining stock: %d\n",
                                userBorrowedBooks, bookStock);

            return true;
        } else {
            System.out.println("Borrow failed! Either stock is insuffic
ient or borrow limit exceeded.");
            return false;
        }
    }

    public int getUserBorrowedBooks() {
        return userBorrowedBooks;
    }

    public int getBookStock() {
        return bookStock;
    }

    public static void main(String[] args) {

        //testcase-VT:
        c03_LibrarySystem library1 = new c03_LibrarySystem(3, 32);
        library1.borrowBook(38); //numBooks > 0 && bookStock > 0 && numBoo
ks > bookStock
        c03_LibrarySystem library2 = new c03_LibrarySystem(2, 20);
        library2.borrowBook(7); //numBooks > 0 && bookStock > 0 && numBook
s <= bookStock && (userBorrowedBooks + numBooks) > maxBorrowLimit
        c03_LibrarySystem library3 = new c03_LibrarySystem(35, 30);
        library3.borrowBook(3); //numBooks > 0 && bookStock > 0 && numBook
s <= bookStock && (userBorrowedBooks + numBooks) <= maxBorrowLimit
        c03_LibrarySystem library4 = new c03_LibrarySystem(10, 30);
        library4.borrowBook(-
3); //numBooks > 0 && bookStock > 0 && numBooks <= bookStock && (userBo
rrowedBooks + numBooks) > maxBorrowLimit

        //testcase-FT:
        // c03_LibrarySystem library1 = new c03_LibrarySystem(2294398, 0);
        // library1.borrowBook(0); //numBooks > 0 && bookStock > 0 && numBo
oks > bookStock
        // c03_LibrarySystem library2 = new c03_LibrarySystem(-
1808425884, 50331648);

```

```

//      Library2.borrowBook(2); //numBooks > 0 && bookStock > 0 && numBo
oks <= bookStock && (userBorrowedBooks + numBooks) > maxBorrowLimit
//      c03_LibrarySystem library3 = new c03_LibrarySystem(-
1808425884, 1677918208);
//      Library3.borrowBook(2); //numBooks > 0 && bookStock > 0 && numBo
oks <= bookStock && (userBorrowedBooks + numBooks) <= maxBorrowLimit
//      c03_LibrarySystem library4 = new c03_LibrarySystem(-
1808425884, 1677918208);
//      Library4.borrowBook(11778); //numBooks > 0 && bookStock > 0 && n
umBooks <= bookStock && (userBorrowedBooks + numBooks) > maxBorrowLimit

//testcase-Z3:
//      c03_LibrarySystem library1 = new c03_LibrarySystem(10, 7);
//      Library1.borrowBook(3);
//      c03_LibrarySystem library2 = new c03_LibrarySystem(10, 4);
//      Library2.borrowBook(5);
//      c03_LibrarySystem library3 = new c03_LibrarySystem(5, 10);
//      Library3.borrowBook(3);
//      c03_LibrarySystem library4 = new c03_LibrarySystem(10, 6);
//      Library4.borrowBook(0);

//      UVT
//      c03_LibrarySystem library1 = new c03_LibrarySystem(10, 7);
//      Library1.borrowBook(38);
//      c03_LibrarySystem library2 = new c03_LibrarySystem(10, 4);
//      Library2.borrowBook(7);
//      c03_LibrarySystem library3 = new c03_LibrarySystem(5, 10);
//      Library3.borrowBook(3);
//      c03_LibrarySystem library4 = new c03_LibrarySystem(10, 6);
//      Library4.borrowBook(-3);

}
}

```

● Coverage report

1. Test case number: 4
2. Vibration testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	810	816
▼ src	0.7 %	6	810	816
▼ code	0.7 %	6	810	816
▼ c03_LibrarySystem.java	100.0 %	6	0	6
▼ c03_LibrarySystem	100.0 %	6	0	6
main(String[])		0	0	0
c03_LibrarySystem(int, int)		0	0	0
borrowBook(int)	100.0 %	6	0	6

3. V-Method testing branch coverage: 94.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	103	15,407	15,510
✓ src	0.7 %	103	15,407	15,510
✓ code	0.7 %	103	15,407	15,510
✓ c03_LibrarySystem.java	94.5 %	103	6	109
✓ c03_LibrarySystem	94.5 %	103	6	109
main(String[])	100.0 %	41	0	41
c03_LibrarySystem(int, int)	100.0 %	12	0	12
borrowBook(int)	100.0 %	50	0	50

4. Fuzz testing branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.5 %	4	812	816
✓ src	0.5 %	4	812	816
✓ code	0.5 %	4	812	816
✓ c03_LibrarySystem.java	66.7 %	4	2	6
✓ c03_LibrarySystem	66.7 %	4	2	6
borrowBook(int)	66.7 %	4	2	6
main(String[])	0.0 %	0	0	0
c03_LibrarySystem(int, int)	0.0 %	0	0	0

5. Fuzz testing instruction coverage: 65.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	71	15,439	15,510
✓ src	0.5 %	71	15,439	15,510
✓ code	0.5 %	71	15,439	15,510
✓ c03_LibrarySystem.java	65.1 %	71	38	109
✓ c03_LibrarySystem	65.1 %	71	38	109
borrowBook(int)	36.0 %	18	32	50
main(String[])	100.0 %	41	0	41
c03_LibrarySystem(int, int)	100.0 %	12	0	12

6. Z3 branch coverage: 83.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.6 %	5	835	840
✓ src	0.6 %	5	835	840
✓ code	0.6 %	5	835	840
✓ c03_LibrarySystem.java	83.3 %	5	1	6
✓ c03_LibrarySystem	83.3 %	5	1	6
borrowBook(int)	83.3 %	5	1	6
main(String[])	0.0 %	0	0	0
c03_LibrarySystem(int, int)	0.0 %	0	0	0

7. Z3 instruction coverage: 94.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	103	21,983	22,086
✓ src	0.5 %	103	21,983	22,086
✓ code	0.5 %	103	21,983	22,086
✓ c03_LibrarySystem.java	94.5 %	103	6	109
✓ c03_LibrarySystem	94.5 %	103	6	109
main(String[])	100.0 %	41	0	41
c03_LibrarySystem(int, int)	100.0 %	12	0	12
borrowBook(int)	100.0 %	50	0	50

8. Unimproved V-method branch coverage: 83.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.6 %	5	835	840
✓ src	0.6 %	5	835	840
✓ code	0.6 %	5	835	840
✓ c03_LibrarySystem.java	83.3 %	5	1	6
✓ c03_LibrarySystem	83.3 %	5	1	6
borrowBook(int)	83.3 %	5	1	6
main(String[])	0.0 %	0	0	0
c03_LibrarySystem(int, int)	0.0 %	0	0	0

9. Unimproved V-method instruction coverage: 94.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓  VibrationTestingTool	 0.5 %	103	21,983	22,086
✓  src	 0.5 %	103	21,983	22,086
✓  code	 0.5 %	103	21,983	22,086
✓  c03_LibrarySystem.java	 94.5 %	103	6	109
✓  c03_LibrarySystem	 94.5 %	103	6	109
 main(String[])	 100.0 %	41	0	41
 c03_LibrarySystem(int, int)	 100.0 %	12	0	12
 borrowBook(int)	 100.0 %	50	0	50